

EvoCharge Backend System Design

Backend architecture, runtime design, and AWS deployment model

Author	Bashir Muhammed
Project	EvoCharge API and AWS Infrastructure
Primary Stack	Java 21, Spring Boot 4, AWS CDK (Java), ECS Fargate, DynamoDB
Document Purpose	Portfolio-grade technical design for backend system engineering review
Version	1.1

1. System Summary

EvoCharge provides a backend platform for charging station discovery, recommendation, operational analytics, and live status updates. The system aggregates operator data into a unified schema and exposes this data through a REST API and SSE stream. The backend is designed for clear service boundaries, managed infrastructure, and predictable runtime behavior under fluctuating traffic.

The core design goal is to keep read operations fast and deterministic while adding optional AI enrichment on advisor endpoints. Production workloads run on Amazon ECS Fargate behind an Application Load Balancer, with DynamoDB as the primary data store.

The backend currently serves as the central integration layer between charging operator datasets and external consumers. It supports search, recommendation, analytics, and advisory responses through one consistent API contract. The implementation prioritizes predictable behavior, clear service ownership, and practical cloud operations.

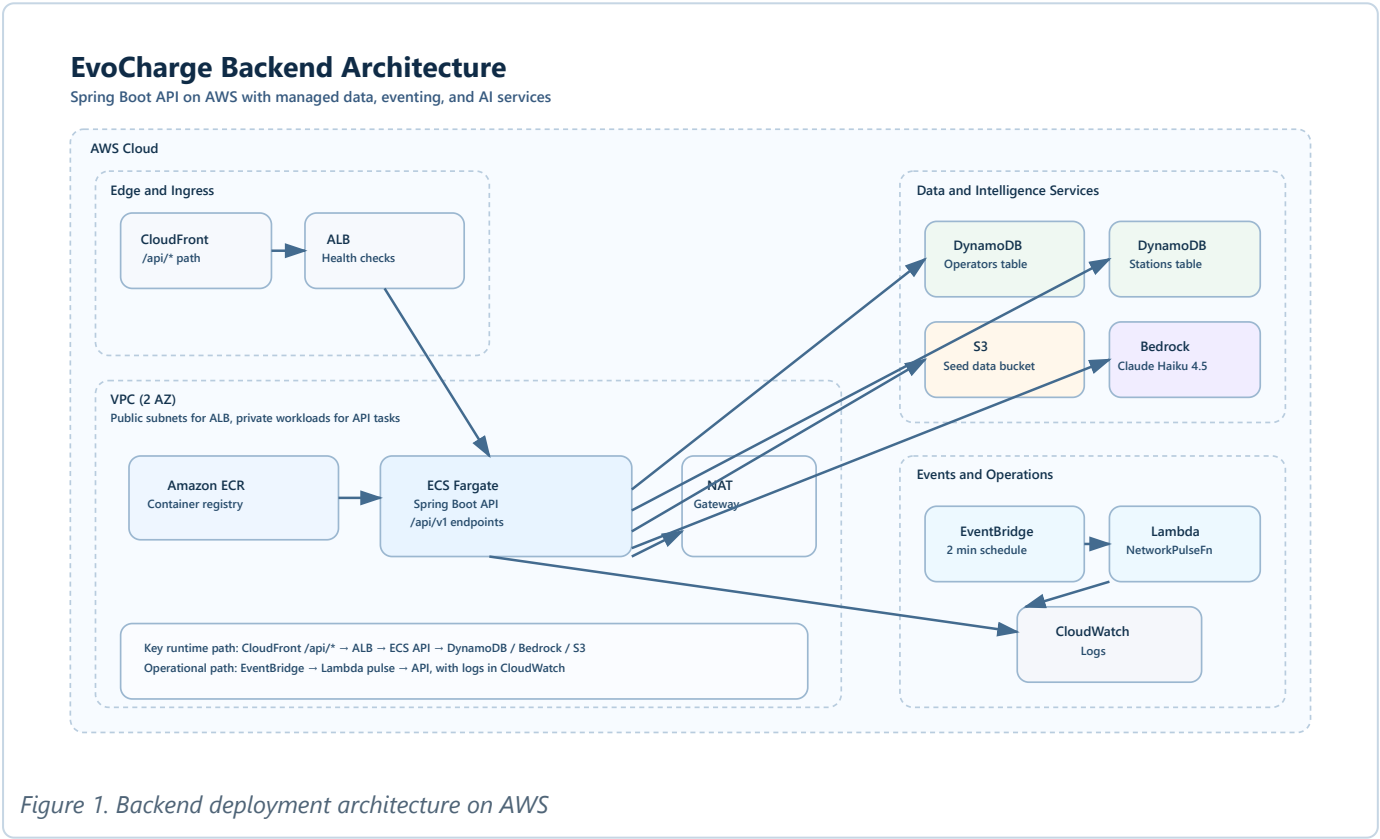
2. Design Goals and Constraints

Area	Design Decision	Rationale
Latency	DynamoDB-backed read paths for stations and operators	Low and stable response time for high-frequency discovery traffic
Scalability	Containerized API on ECS Fargate	Horizontal scaling without host management
Reliability	ALB health checks plus stateless API tasks	Automatic replacement of unhealthy tasks
Maintainability	AWS CDK stack separation (Network, Data, API, Web edge)	Safer deployments and clearer ownership boundaries
AI Integration	Bedrock invoked only on advisor endpoint	Core API routes stay deterministic and cost controlled
Operational Simplicity	Managed AWS services over self-hosted components	Reduced maintenance effort and faster release velocity

Target Non Functional Profile

Category	Target	Notes
Read latency	P95 under 300 ms for discovery endpoints	DynamoDB query paths and lightweight API transformations
Availability	Service continuity with automatic task replacement	ALB health checks and ECS self-healing behavior
Recovery behavior	Graceful degradation for advisor feature	Fallback response when Bedrock call fails
Deployment safety	Zero to low interruption rollout	Task definition revisions and forced rolling deployment

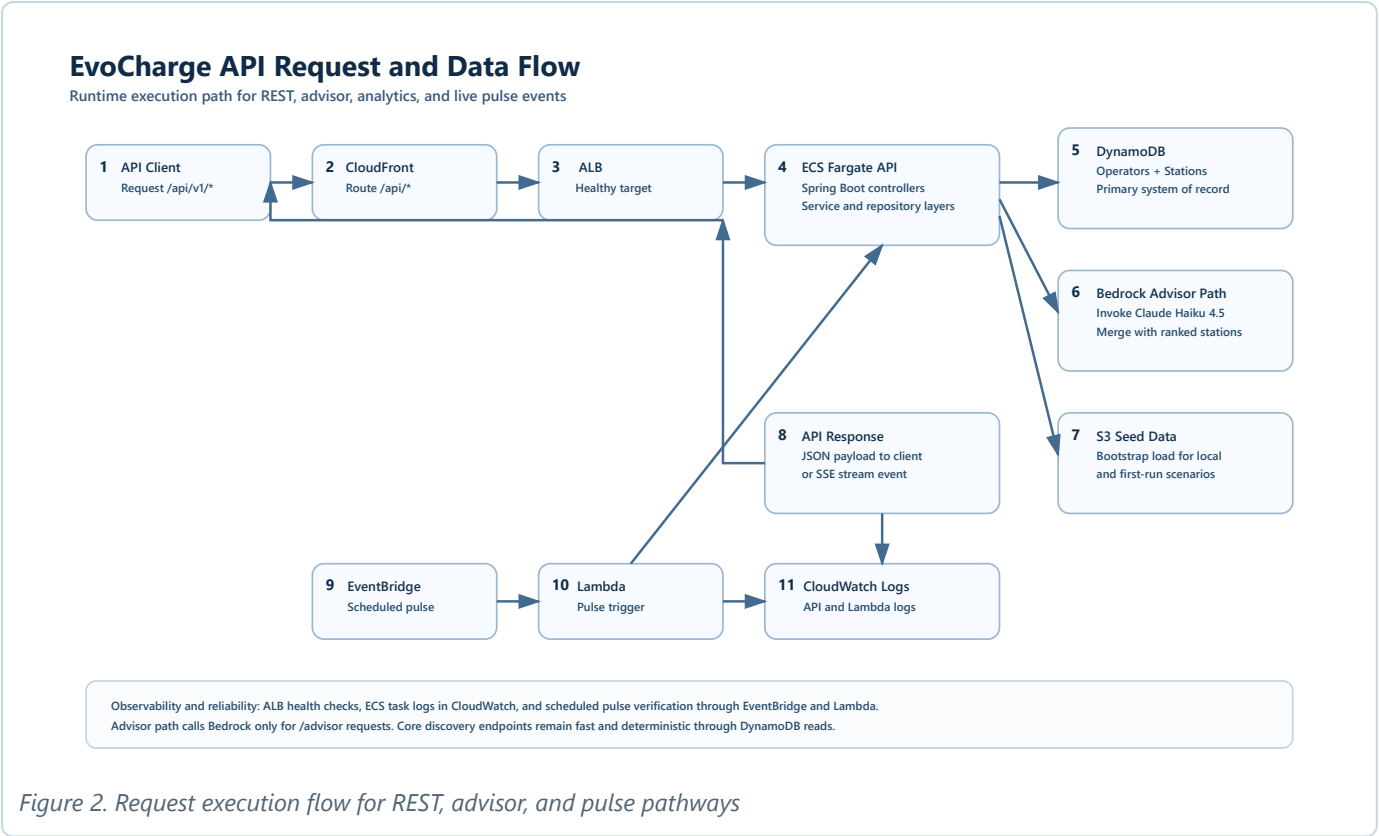
3. High Level Backend Architecture



Component Notes

- **CloudFront and ALB:** CloudFront handles edge routing for `/api/*` to ALB. ALB performs health checks on API tasks.
- **ECS Fargate API:** Spring Boot service hosts REST endpoints and SSE stream endpoints.
- **DynamoDB:** Separate operators and stations tables support filtered discovery and analytical aggregation.
- **Bedrock:** Advisor endpoint composes station context and calls Claude Haiku 4.5 for natural-language responses.
- **EventBridge and Lambda:** Scheduled pulse path drives status update simulation and stream freshness checks.
- **CloudWatch:** Centralized logs for API and Lambda diagnostics.

4. Request and Data Flow Design



Endpoint Processing Model

Endpoint Group	Primary Path	Secondary Path
/operators, /stations, /nearby	Controller → Service → DynamoDB	Optional seed bootstrap if storage is not initialized
/recommend	Controller → EvoScore service → station ranking	None
/advisor	Recommendation path + Bedrock invocation	Template fallback response when Bedrock fails
/events/stream	SSE subscription and status events	EventBridge and Lambda pulse keep live signal cadence

Detailed Request Lifecycle

Step	Description	Outcome
Ingress	Client request reaches CloudFront and is routed to ALB for /api/*.	Request enters protected compute path.
Routing	ALB forwards only to healthy ECS tasks.	Failed tasks are isolated automatically.
Domain execution	Controller validates input, service layer executes business logic.	Consistent API behavior and reusable logic paths.

Data operations	Repository reads or writes through DynamoDB tables.	Low latency access for station and operator datasets.
AI enrichment	Advisor endpoint calls Bedrock and merges textual advisory output.	Readable recommendation explanation for users.
Response	JSON or SSE payload returns through ALB and CloudFront.	Stable contract for external clients.

5. API and Domain Design

Primary Endpoint Surface

Domain	Endpoint Group	Purpose
Health	/api/v1/health	Readiness and liveness signal for load balancer and monitoring.
Discovery	/operators, /stations, /nearby	Filtered charging station discovery and lookup flows.
Recommendation	/recommend	EvoScore ranking based on user context and station quality factors.
Analytics	/analytics/summary, /analytics/demand-by-area	Operational and planning metrics from backend data.
Live updates	/events/stream	SSE channel for real-time status notifications.
Advisor	/advisor	Natural language recommendation assistant with Bedrock integration.

Service Layer Structure

Core services

- StationService for listing, filtering, and proximity search.
- RecommendationService and EvoScoreService for deterministic ranking.
- AnalyticsService for summary and demand aggregation.

Supporting services

- AdvisorService for Bedrock prompt orchestration and fallback handling.
- Pulse and event services for SSE emission and periodic status changes.
- Seeder service for controlled startup data initialization.

6. Data Model and Access Strategy

Core Entities

Entity	Purpose	Key Fields
Operator	Represents charging network owner	id, name, contact metadata

Station	Represents physical charging location	id, city, area, status, connectors, waitMinutes, reliabilityScore
Recommendation	Runtime ranking output	distanceKm, evoScore, station reference
Status Event	Live update payload for SSE clients	stationId, status, timestamp

EvoScore Logic

Recommendation ranking is weighted and deterministic. Distance, availability, wait time, reliability, and connector compatibility are normalized and combined into a single score. This keeps recommendation behavior explainable and stable across requests.

Current weights are tuned for practical decision quality: distance (30%), availability (25%), wait time (20%), reliability (15%), and connector match (10%). This model favors nearby and immediately usable stations while still respecting station quality.

DynamoDB Access Patterns

Access Pattern	Typical Query	Optimization Notes
Station detail fetch	Get by station id	Primary key lookup for low latency deterministic reads.
Filtered station list	City, operator, status, connector filters	Service-side composition and ranking from normalized records.
Nearby search	Lat/Ing and radius input	Distance computed in service layer after candidate retrieval.
Demand analytics	Area-level grouped summary	Aggregation path optimized for dashboard reads.

7. Security and Compliance Posture

Control Area	Implementation	Risk Addressed
Identity and access	IAM task role with scoped permissions for DynamoDB, S3, Bedrock, and Location.	Prevents broad credential exposure and privilege escalation.
Network boundary	ALB ingress with ECS tasks in VPC and NAT managed egress.	Reduces direct exposure of runtime containers.
Secret handling	No static cloud keys in source or container images.	Avoids credential leakage through code distribution channels.
Transport security	CloudFront edge and controlled API ingress route.	Protects client traffic and standardizes request pathing.
Auditability	CloudWatch logs for API and Lambda execution flows.	Supports incident investigation and runtime verification.

8. Reliability, Failure Handling, and Observability

Failure Behavior Matrix

Failure Scenario	Current Behavior	User Impact	Mitigation
Unhealthy API task	ALB removes task from target pool	Minimal if healthy tasks remain	ECS launches replacement task automatically
Bedrock call failure	Advisor service returns fallback message	Advisor quality reduced, core API unaffected	Retry strategy and fallback text path
DynamoDB throttling or errors	Endpoint returns controlled error	Data retrieval delayed or failed for specific call	Capacity tuning and request optimization
Pulse scheduler interruption	SSE cadence drops temporarily	Live status updates slow down	EventBridge schedule and Lambda monitoring

Observability Stack

Observability is built around CloudWatch logs and health endpoints. API logs cover request lifecycle and downstream service calls. Lambda logs cover pulse triggers and API reachability. ALB health checks provide a direct availability signal for task routing.

9. Operational Architecture

Concern	Current Implementation	Production Benefit
Deployments	CDK provision + ECR image rollout to ECS task revisions	Repeatable release process and traceable changes
Health and readiness	ALB health check on /api/v1/health	Only healthy tasks receive traffic

Observability	CloudWatch logs for API and Lambda	Fast root cause analysis during incidents
Security model	IAM task role with scoped service permissions	No long-lived credentials in code or containers
Scalability path	Stateless API tasks and managed services	Straightforward horizontal scaling
Release process	Docker image build, ECR push, ECS task revision update	Clear rollback by switching to prior task definition revision
Infrastructure evolution	CDK controlled stack updates	Repeatable environment changes with source-controlled IaC

Operational Runbook Summary

Operation	Primary Action	Verification
Deploy new API image	Build image, push to ECR, update ECS service with new task revision	Health endpoint and ECS service stability checks
Rollback deployment	Revert ECS service to previous task definition revision	Traffic recovery and error-rate normalization
Investigate incident	Inspect CloudWatch logs, ECS task events, and ALB target health	Root cause identified and corrective action applied

10. Cost and Scalability Considerations

Dimension	Current State	Scale Strategy
Compute cost	Single Fargate service with right-sized CPU and memory	Scale tasks by traffic profile and tune resource class
Database cost	DynamoDB on-demand mode	Move to provisioned with auto scaling for stable high traffic
AI cost	Bedrock invoked on advisor endpoints only	Rate-limit advisor path and cache repeated advisory context
Network and edge	ALB plus CloudFront API path routing	Introduce tighter cache policy for non-personalized responses

11. Future Technical Roadmap

Authentication and tenant isolation	Operator ingestion connectors	Formal API versioning policy
Automated load and chaos testing	Distributed tracing integration	SLO dashboards and alert rules

The next evolution is to replace seed ingestion with authenticated operator pipelines and enforce tenant-aware data boundaries. On the platform side, introducing tracing, strict SLOs, and automated performance gates will further improve reliability under enterprise traffic.

12. Engineering Evaluation

This backend is built with practical production patterns. It cleanly separates ingress, compute, data, events, and AI services. The API path remains efficient for core data retrieval while advanced advisor responses are isolated to dedicated endpoint logic. The system is ready for real operator feed integration by replacing seed ingestion with authenticated upstream connectors.

From a hiring perspective, this project demonstrates strong backend ownership across service design, cloud infrastructure, deployment automation, and runtime operations. The architecture choices are grounded in maintainability and service reliability rather than novelty. The implementation is intentionally straightforward to support long-term handover and team extension.

Backend Skills Demonstrated

- Java and Spring Boot service architecture with clean endpoint grouping.
- AWS infrastructure design using ECS, ALB, DynamoDB, EventBridge, Lambda, and Bedrock.
- Operational delivery with containerized deployment and revision-based rollouts.
- Resilience planning with controlled fallback behavior and health-driven routing.
- Production thinking through observability, security scope, and scalability planning.